

Лабораторная работа

Тема

Создание проекта с применением открытых публичных REST-сервисов и тестирование бизнес-процесса в SoapUI.

Цель работы

Целью лабораторной работы является изучение принципов работы REST-сервисов, создание проекта в SoapUI, выполнение GET- и POST-запросов к открытым публичным API, а также реализация тестового сценария TestCase для проверки бизнес-процесса.

Используемые инструменты

В ходе выполнения лабораторной работы использовались следующие инструменты:

- SoapUI 5.1.0;
- открытый REST-сервис JSONPlaceholder;
- открытый REST-сервис Open-Meteo;
- протокол HTTP;
- формат передачи данных JSON.

Описание бизнес-процесса

Для выполнения лабораторной работы был разработан бизнес-процесс оформления заявки на доставку с предварительной проверкой данных клиента и погодных условий.

Сценарий бизнес-процесса:

1. Получение информации о клиенте по его идентификатору.
2. Проверка погодных условий в точке доставки.
3. Создание тестовой заявки на доставку.
4. Проверка успешного выполнения запросов с помощью Assertions в SoapUI.
5. Запуск тестового сценария TestCase.

Данный процесс имитирует работу информационной системы доставки, где перед созданием заявки необходимо получить данные клиента и проверить внешние условия, которые могут повлиять на выполнение заказа.

Используемые REST-сервисы

JSONPlaceholder

JSONPlaceholder использовался для получения данных пользователя и создания тестовой заявки.

Использованные запросы:

```
GET https://jsonplaceholder.typicode.com/users/1
```

```
POST https://jsonplaceholder.typicode.com/posts
```

Open-Meteo

Open-Meteo использовался для получения информации о текущей погоде по координатам.

Использованный запрос:

```
GET https://api.open-meteo.com/v1/forecast?
latitude=52.52&longitude=13.41&current=temperature_2m,wind_speed_10m,precipitation&timezone=auto
```

Ход выполнения работы

1. Создание REST-проекта в SoapUI

В программе SoapUI был создан новый REST-проект. В качестве первого адреса был указан открытый REST-сервис JSONPlaceholder:

```
https://jsonplaceholder.typicode.com/users/1
```

После создания проекта в дереве SoapUI появился ресурс, содержащий GET-запрос к сервису.

2. Выполнение GET-запроса для получения данных клиента

На первом этапе был выполнен GET-запрос:

```
GET https://jsonplaceholder.typicode.com/users/1
```

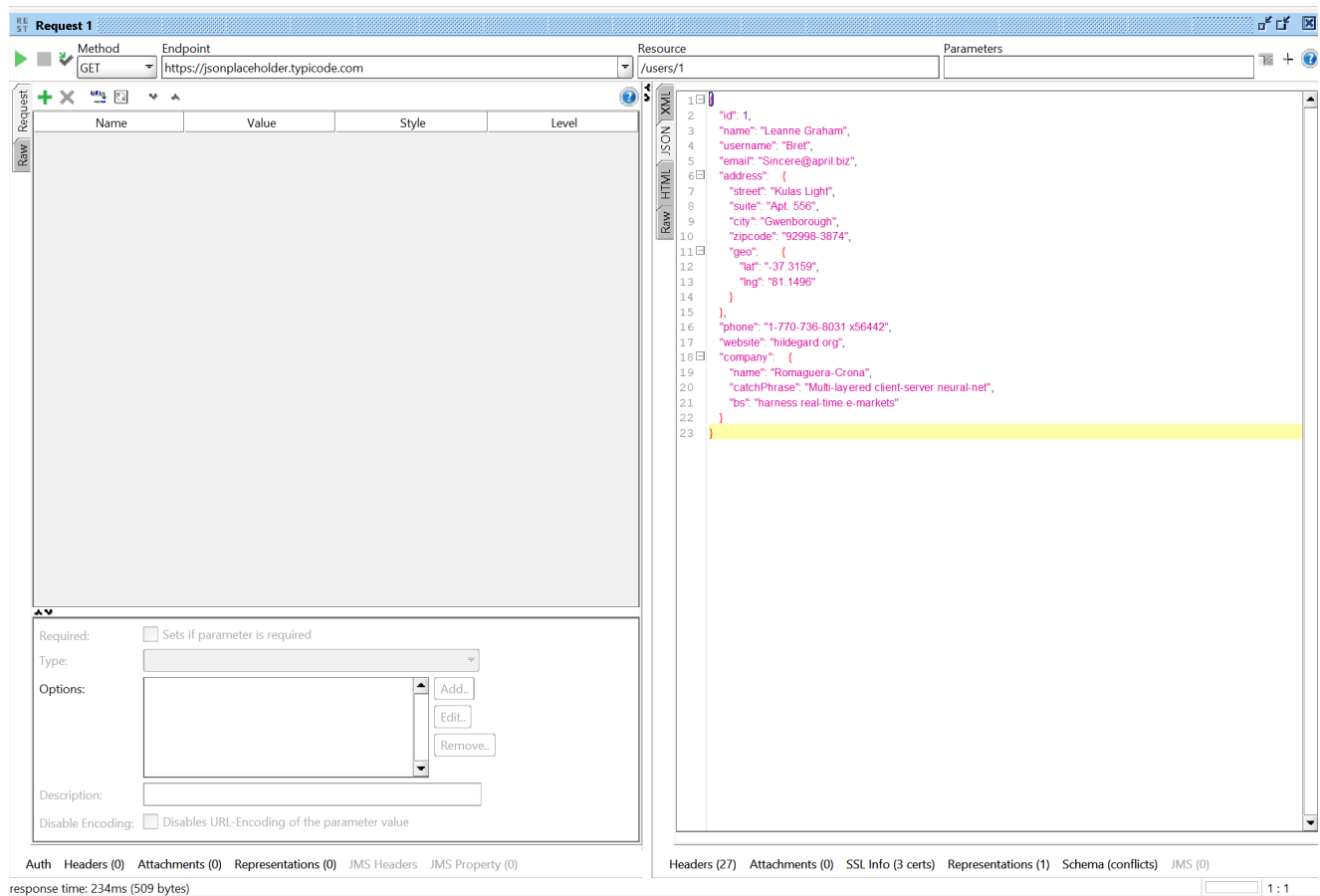
В результате выполнения запроса сервер вернул ответ в формате JSON. В ответе содержались данные пользователя: идентификатор, имя, логин, электронная почта, адрес, телефон, сайт и информация о компании.

Для данного запроса были добавлены проверки Assertions:

- проверка HTTP-статуса ответа 200;

- проверка наличия идентификатора пользователя;
- проверка наличия поля email;
- проверка времени ответа через Response SLA.

Все добавленные проверки были выполнены успешно.



3. Выполнение GET-запроса для проверки погодных условий

На следующем этапе был добавлен REST-запрос к сервису Open-Meteo. Запрос использовался для получения текущих погодных условий по координатам.

Использованный запрос:

```

GET https://api.open-meteo.com/v1/forecast?
latitude=52.52&longitude=13.41&current=temperature_2m,wind_speed_10m,precipitation&timezone=auto

```

В ответе были получены погодные данные в формате JSON. Ответ содержал координаты, часовой пояс, единицы измерения и блок `current` с текущими значениями температуры, скорости ветра и осадков.

Для запроса были добавлены следующие проверки:

- проверка HTTP-статуса ответа 200;
- проверка наличия поля `current`;
- проверка наличия поля `temperature_2m`;

- проверка наличия поля wind_speed_10m;
- проверка времени ответа через Response SLA.

Все проверки были выполнены успешно.

Request 1

Method: GET Endpoint: https://api.open-meteo.com Resource: /v1/forecast Parameters: ?latitude=52.52&longitude=13.41¤t=temperature_2m,wind_speed_10m,precipitation&timezone=auto

Name	Value	Style	Level
latitude	52.52	QUERY	RESOURCE
longitude	13.41	QUERY	RESOURCE
current	temperature_2m,wind_spee...	QUERY	RESOURCE
timezone	auto	QUERY	RESOURCE

Required: ☐ Sets if parameter is required

Type:

Options: Add... Edit... Remove...

Description:

Disable Encoding: ☐ Disables URL-Encoding of the parameter value

Auth Headers (0) Attachments (0) Representations (0) JMS Headers JMS Property (0)

response time: 383ms (423 bytes)

```

1 {
2   "latitude": 52.52,
3   "longitude": 13.419998,
4   "generationtime_ms": 0.09608268737792969,
5   "utc_offset_seconds": 7200,
6   "timezone": "Europe/Berlin",
7   "timezone_abbreviation": "GMT+2",
8   "elevation": 38,
9   "current_units": {
10    "time": "iso8601",
11    "interval": "seconds",
12    "temperature_2m": "°C",
13    "wind_speed_10m": "km/h",
14    "precipitation": "mm"
15  },
16  "current": {
17    "time": "2026-06-23T02:15",
18    "interval": 900,
19    "temperature_2m": 18.8,
20    "wind_speed_10m": 5.2,
21    "precipitation": 0
22  }
23 }

```

Headers (5) Attachments (0) SSL Info (2 certs) Representations (2) Schema (conflicts) JMS (0)

Forecast 1 - Request 1

Endpoint: https://api.open-meteo.com

Resource/Method: Forecast -> Forecast 1 [/v1/forecast?latitude=52.52&longitude=13.41¤t=temperature_2m,wind_speed_10m,precipitation&timezone=auto]

Name	Value	Style	Level
latitude	52.52	QUERY	RESOURCE
longitude	13.41	QUERY	RESOURCE
current	temperature_2m,wind_spee...	QUERY	RESOURCE
timezone	auto	QUERY	RESOURCE

Auth Headers (0) Attachments (0) Representations (0) JMS Headers JMS Property (0)

```

1 {
2   "latitude": 52.52,
3   "longitude": 13.419998,
4   "generationtime_ms": 0.057578086853027344,
5   "utc_offset_seconds": 7200,
6   "timezone": "Europe/Berlin",
7   "timezone_abbreviation": "GMT+2",
8   "elevation": 38,
9   "current_units": {
10    "time": "iso8601",
11    "interval": "seconds",
12    "temperature_2m": "°C",
13    "wind_speed_10m": "km/h",
14    "precipitation": "mm"
15  },
16  "current": {
17    "time": "2026-06-23T02:30",
18    "interval": 900,
19    "temperature_2m": 18.8,
20    "wind_speed_10m": 4.7,
21    "precipitation": 0
22  }
23 }

```

Headers (5) Attachments (0) SSL Info (2 certs) Representations (2) Schema (conflicts) JMS (0)

Online Help

Valid HTTP Status Codes - VALID

Contains - VALID

Contains 1 - VALID

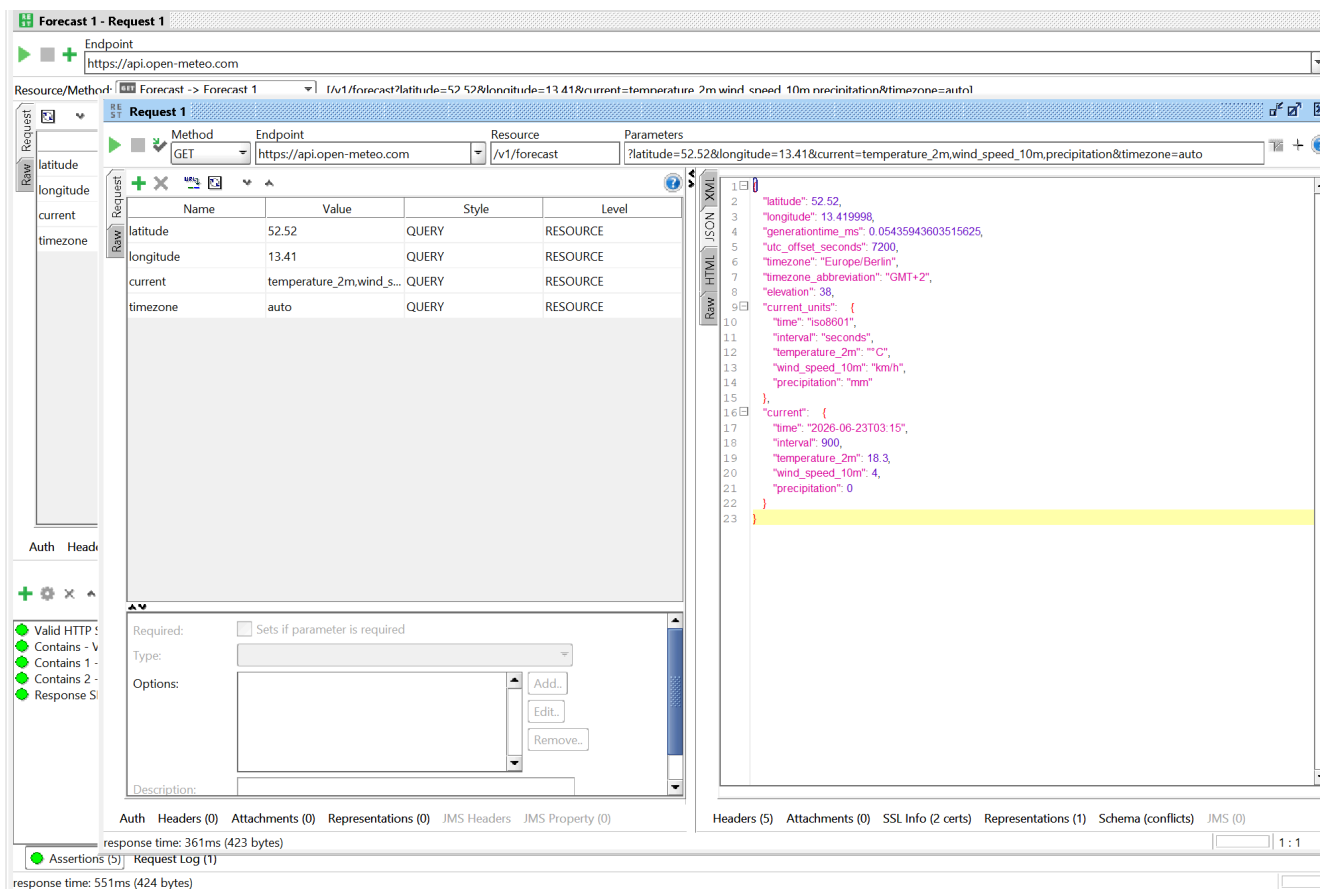
Contains 2 - VALID

Response SLA - VALID

Assertions (5) Request Log (1)

response time: 551ms (424 bytes)

SoapUI loa http loa iettv loa error loa wsm loa memory loa



4. Выполнение POST-запроса для создания заявки

После проверки данных клиента и погодных условий был выполнен POST-запрос к сервису JSONPlaceholder для создания тестовой заявки на доставку.

Использованный запрос:

```
POST https://jsonplaceholder.typicode.com/posts
```

В тело запроса были переданы данные в формате JSON:

```
{
  "userId": 1,
  "title": "Delivery order",
  "body": "Create delivery order after customer and weather validation"
}
```

После выполнения запроса сервер вернул ответ в формате JSON. В ответе содержались переданные данные заявки и созданный идентификатор записи.

Для POST-запроса были добавлены проверки:

- проверка HTTP-статуса ответа 201;
- проверка наличия поля id;
- проверка наличия userId;

- проверка наличия текста Delivery order;
- проверка времени ответа через Response SLA.

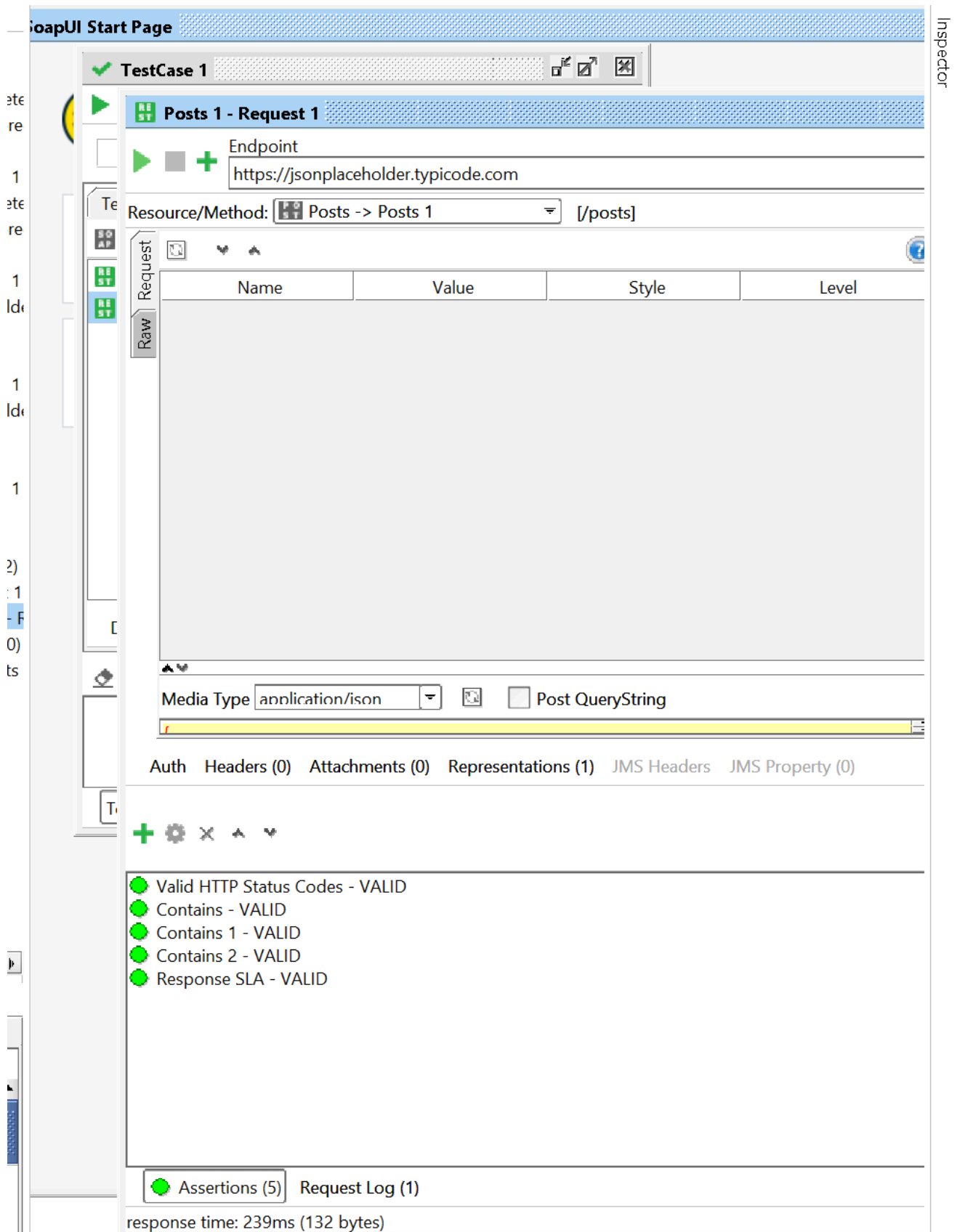
Все проверки были выполнены успешно.

The screenshot displays a REST client interface with the following components:

- Top Bar:** Includes links for Forum, Trial, Preferences, Proxy, and Online Help.
- Request Configuration:**
 - Method:** POST
 - Endpoint:** https://jsonplaceholder.typicode.com
 - Resource:** /posts
 - Parameters:** (Empty)
- Request Body:** A table with columns Name, Value, Style, and Level. Below it, a 'Required' checkbox is set to 'Sets if parameter is required'. The 'Media Type' is set to 'application/json'.
- Request Payload (JSON):**

```
{
  "userId": 1,
  "title": "Delivery order",
  "body": "Create delivery order after customer and weather validation",
  "id": 101
}
```
- Response Panel:** Shows the received JSON response, which matches the request payload exactly:

```
{
  "userId": 1,
  "title": "Delivery order",
  "body": "Create delivery order after customer and weather validation",
  "id": 101
}
```
- Bottom Tabs:** Auth, Headers (0), Attachments (0), Representations (1), JMS Headers, JMS Property (0) on the left; Headers (28), Attachments (0), SSL Info (3 certs), Representations (1), Schema (conflicts), JMS (0) on the right.



5. Создание и запуск TestCase

Для автоматизированной проверки бизнес-процесса в SoapUI был создан TestCase. В него были добавлены REST-запросы, связанные с выполнением бизнес-процесса:

1. Проверка погодных условий через сервис Open-Meteo.
2. Создание заявки через сервис JSONPlaceholder.

После запуска TestCase оба тестовых шага были выполнены успешно. В окне SoapUI отображился статус FINISHED, а в журнале выполнения были показаны успешные результаты выполнения шагов.

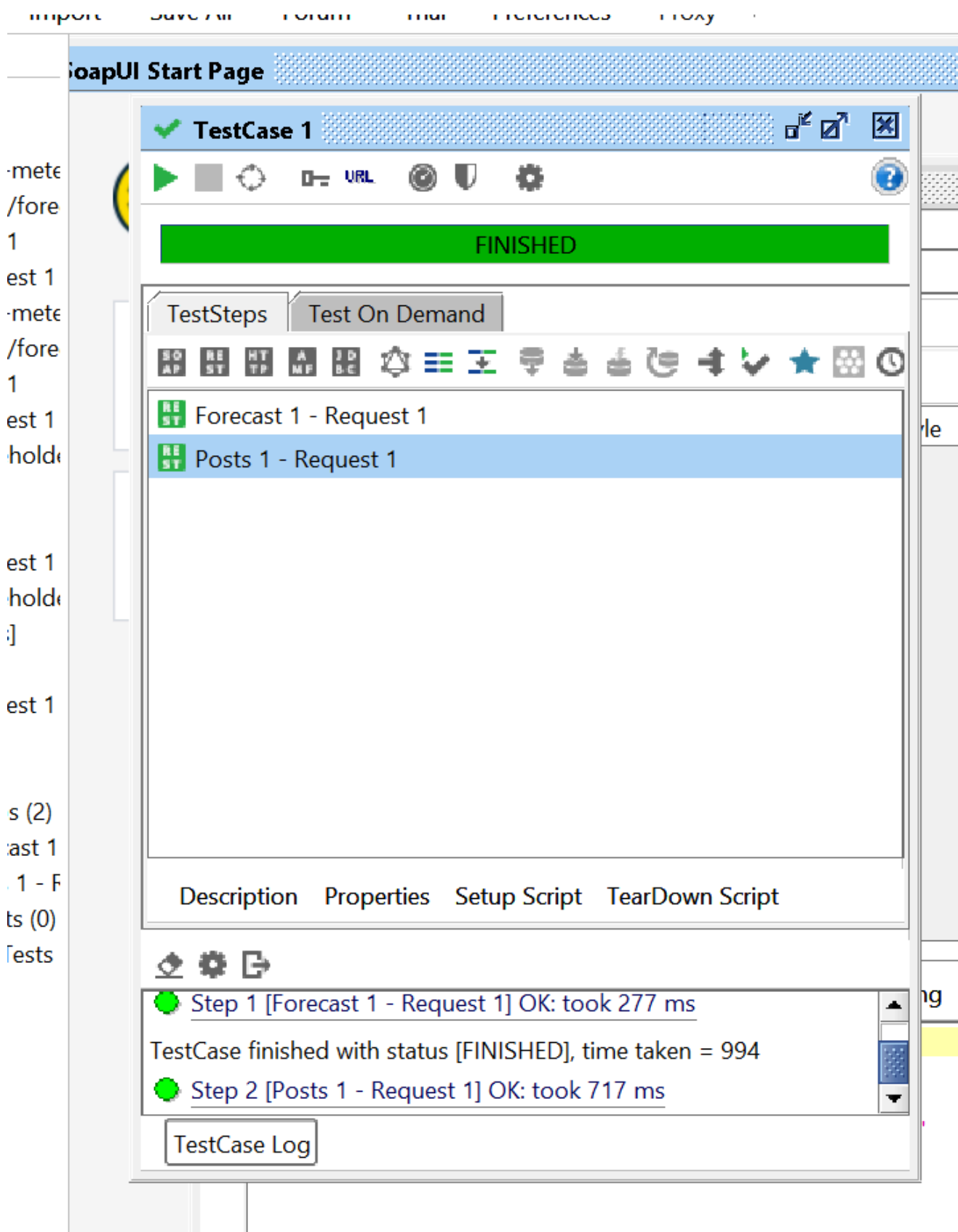


Таблица тестовых шагов

№	Название шага	Метод	REST-сервис	Ожидаемый результат
1	Получение данных клиента	GET	JSONPlaceholder	Получен JSON-ответ с данными пользователя
2	Проверка погоды	GET	Open-Meteo	Получен JSON-ответ с текущей погодой
3	Создание заявки	POST	JSONPlaceholder	Создана тестовая заявка, получен ответ с id

Результаты тестирования

В результате выполнения лабораторной работы были успешно отправлены GET- и POST-запросы к открытым публичным REST-сервисам. Для каждого запроса были настроены проверки Assertions, позволяющие убедиться в корректности HTTP-статуса, наличии нужных данных в JSON-ответе и допустимом времени ответа.

Запрос к JSONPlaceholder вернул данные пользователя. Запрос к Open-Meteo вернул погодные данные. POST-запрос к JSONPlaceholder успешно создал тестовую заявку. Итоговый запуск TestCase завершился успешно со статусом FINISHED.

Вывод

В ходе лабораторной работы был создан проект в SoapUI с применением открытых публичных REST-сервисов. Был разработан бизнес-процесс оформления заявки на доставку, включающий получение данных клиента, проверку погодных условий и создание заявки. Для REST-запросов были настроены проверки Assertions, подтверждающие корректность выполнения запросов. Созданный TestCase позволил автоматически проверить выполнение бизнес-процесса. Все тестовые шаги были выполнены успешно.